AI ACADEMY · LEVEL 6 · GRADE 11 · WEEK 16 · STUDENT LESSON

# Timeouts retries and cancellation

Learn to understand, design, build, test and explain AI systems responsibly.

Project: Grounded Learning Service

THE CONNECTED WEEK

## Bound the wait. Check the retry. Respect cancellation.

Calculate conditional caller-wait bounds and distinguish finite attempts, failure categories, cancellation and uncertain remote effects.

---

**Keep one habit:** Timing contract → labelled timeline → complete event table → separate cancellation picture → numeric/stop probes → virtual simulator → independent bound.

MAIN LESSON · SECTION 1

## Bound the caller’s wait and decide whether another attempt is permitted

> Timeout, retry budget and cancellation control different parts of the process.

### Learning goals

I can explain the **mechanism** and terminology of **timeout**s retries and **cancellation**.

I can apply the stated procedure to a new case with a traceable result.

I can identify an assumption or limitation and design a revealing follow-up.

## Retrieve uncertain effects and duplicate protection

Close your notes first. A rough first answer helps us choose the right starting point; it is not a score for your ability.

Without opening the old lesson, explain one key idea from Approval gates and idempotency. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from State machines for bounded agents. Give an example and a mistake that the idea helps you avoid.

Without opening the old lesson, explain one key idea from Evaluation pipelines. Give an example and a mistake that the idea helps you avoid.

After trying, check the relevant lesson or ask your teacher. Change the part of your explanation that the **evidence** shows was wrong.

## Prerequisite and starting response

- Recall Approval gates and idempotency. Explain why a recorded result needs its **input** conditions and **evidence** source before it can support a claim.

> **My starting explanation**
>
> _____
>
> _____
>
> _____
>
> _____

## End goal

Explain **timeout**s retries and **cancellation** using the deciding step in this fictional service.

▼ **Earlier wording, alternatives and picture transcripts**

**Your learning target**

**Remember before reading**

This week you will practise:

Explain the **mechanism** and terminology of **timeout**s retries and **cancellation**

Apply the stated procedure to a new case with a traceable result

Identify an assumption or limitation and design a revealing follow-up

**Retrieve what you already know**

**This week’s end goal**

---

MAIN LESSON · SECTION 2

# Define the retry policy before adding up durations

> A timeout by itself does not prove either safe retry or remote cancellation.

## Begin with caller uncertainty and future work

A caller can stop waiting for an answer without knowing whether the remote operation stopped. A retry is another attempt, which needs permission and a remaining budget. Cancellation tells the local process not to start more work; it does not automatically undo an effect already committed elsewhere. First define which failures may retry and exactly what each timeout bounds.

Our local example uses virtual time and supplied outcomes, with no real waiting or remote operation.

## Define timeout, retry and cancellation

**Timeout**s bound waiting; retries consume a finite budget. **Cancellation** should stop future work while accounting for operations already committed.

## Complete attempts, caps, gaps and failure policy

| Main virtual waiting contract | Declared value or rule |
|---|---|
| Maximum attempts | 3 total attempts, including the first; at most 2 retries |
| Per-attempt cap | Caller waits at most 4 seconds under the enforced local model |
| Retry gaps | 1 second before attempt 2; 2 seconds before attempt 3 |
| Retryable categories | Only timeout/transient under this declared simulation policy |
| Nonretryable categories | Authentication, permission or invalid-request failures stop automatic retry |
| Cancellation model | Can be supplied before an attempt/backoff starts; prevents that future attempt |
| Overhead | Excluded from arithmetic; no real timer is used |
| Side-effect scope | Read-only model or a separately justified safe-retry policy; no automatic duplicate/undo guarantee |
| Maximum modeled waiting | 3×4 +1+2 =15 seconds; no fourth attempt |

## Name supplied and missing evidence

**List the given facts and one fact that is not supplied.**

▼ **Earlier wording, alternatives and picture transcripts**

Begin by writing the question the procedure is intended to answer. Identify the information already supplied, the information it will produce and the information still missing.

These are constructed classroom examples; they do not require live accounts, private records or an external service. Do not treat a plausible explanation as a measured outcome.

MAIN LESSON · SECTION 3

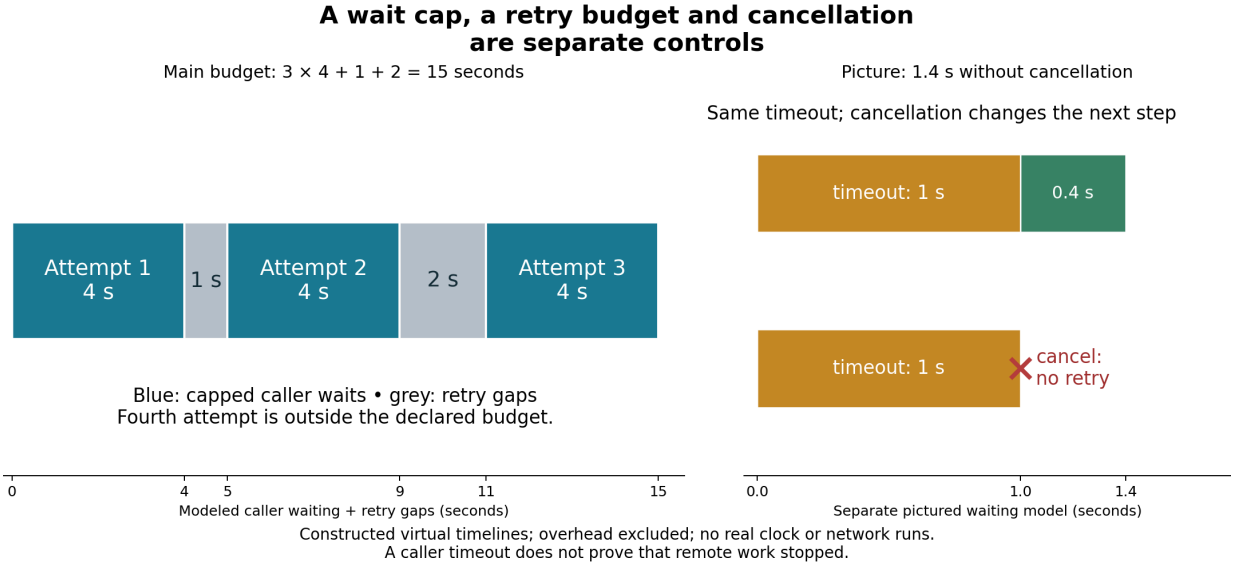
# Trace each wait and gap to the budget boundary

> The 15-second total is conditional on enforced caps and excludes overhead.

## Original transient-failure and wait-bound calculation

Retry only declared transient failures. Use at most three attempts with waiting gaps 1 and 2 seconds; the caller waits at most 4 seconds for each attempt under the enforced local timeout model. Worst-case elapsed bound excluding overhead is 3*4+1+2=15 seconds.

## Labelled main and pictured timelines



### Complete event-by-event main arithmetic

| Main event | Modeled interval in seconds | Elapsed end | Reason |
|---|---|---|---|
| Attempt1 times out | 0–4 | 4 | Uses first capped caller wait |
| Gap before retry | 4–5 | 5 | Declared1 second gap |
| Attempt2 times out | 5–9 | 9 | Second of3 attempts |
| Gap before retry | 9–11 | 11 | Declared2 second gap |
| Attempt3 times out | 11–15 | 15 | Budget exhausted |
| Proposed attempt 4 | Not started | 15 | An extra desired result does not add authority or budget |

## State units and timing assumptions

Before applying the procedure, write its units, denominator or decision boundary where relevant. A partner should be able to repeat each step using the same **input**s.

If a required **input** is absent or violates the stated contract, record that condition explicitly instead of quietly replacing it with a convenient value.

## Explain the visual

Point to the **input**, the deciding step and the result in this week's visual. Explain the same step in ordinary words. A picture helps when you can say what each part represents.

▼ Earlier wording, alternatives and picture transcripts

### Timeouts retries and cancellation — evidence trace

| INPUT AND SCOPE | CHECK OR CALCULATION | CONCLUSION AND LIMIT |
| --- | --- | --- |
| Three attempts each time out after 4 seconds with gaps 1 and 2. A fourth is proposed. | Retry only declared transient failures. Use at most three attempts with waiting gaps 1 and 2 seconds; the caller waits at most 4 seconds for each attempt under the enforced local timeout model. Worst-case elapsed bound excluding overhead is 3*4+1+2=15 seconds. | Budget is exhausted after 15 seconds under the model; reject fourth attempt. A late side-effect result must still be reconciled. |

The figure connects this chapter's worked input, declared procedure and result. Read the complete worked reasoning for qualifiers and intermediate steps; the abbreviated labels do not replace that reasoning.

**Connect the picture and the explanation**

---

MAIN LESSON · SECTION 4

## Change the next-action condition without rewriting earlier waits

> An unused retry allowance does not override cancellation.

## Current task and response

- Three attempts each time out after 4 seconds with gaps 1 and 2. A fourth is proposed. Show the intermediate steps and give the conclusion with its scope.

> **Keep the procedure fixed. Explain what changed in the inputs and result.**

## Predict cancellation after the pictured timeout

- Change: **Cancellation** arrives after the first **timeout**.

▼ **Check after predicting**

**Prediction**: Do not start the second attempt.

Reason: **Cancellation** overrides the unused retry allowance.

▼ **Earlier wording, alternatives and picture transcripts**

**Compare a familiar case with a changed case**

**Change one thing in the pictured case**

---

MAIN LESSON · SECTION 6

# Compare the same first timeout with different next decisions

> Keep the main 3-attempt budget separate from the pictured 2-attempt case.

## Main three-timeout scenario and no-fourth conclusion

Three attempts each time out after 4 seconds with gaps 1 and 2. A fourth is proposed.

Budget is exhausted after 15 seconds under the model; reject fourth attempt. A late side-effect result must still be reconciled.

## Keep the trace's scope

Compare the intermediate **evidence** with the final statement. Explain why each number or decision follows from the stated **input**s.

The conclusion belongs to this scenario and procedure; changing the **input**s, evaluation population or authority can change what it establishes. Retain the raw record so a later revision can be compared honestly.

## Separate picture caps, success fixture and cancellation boundary

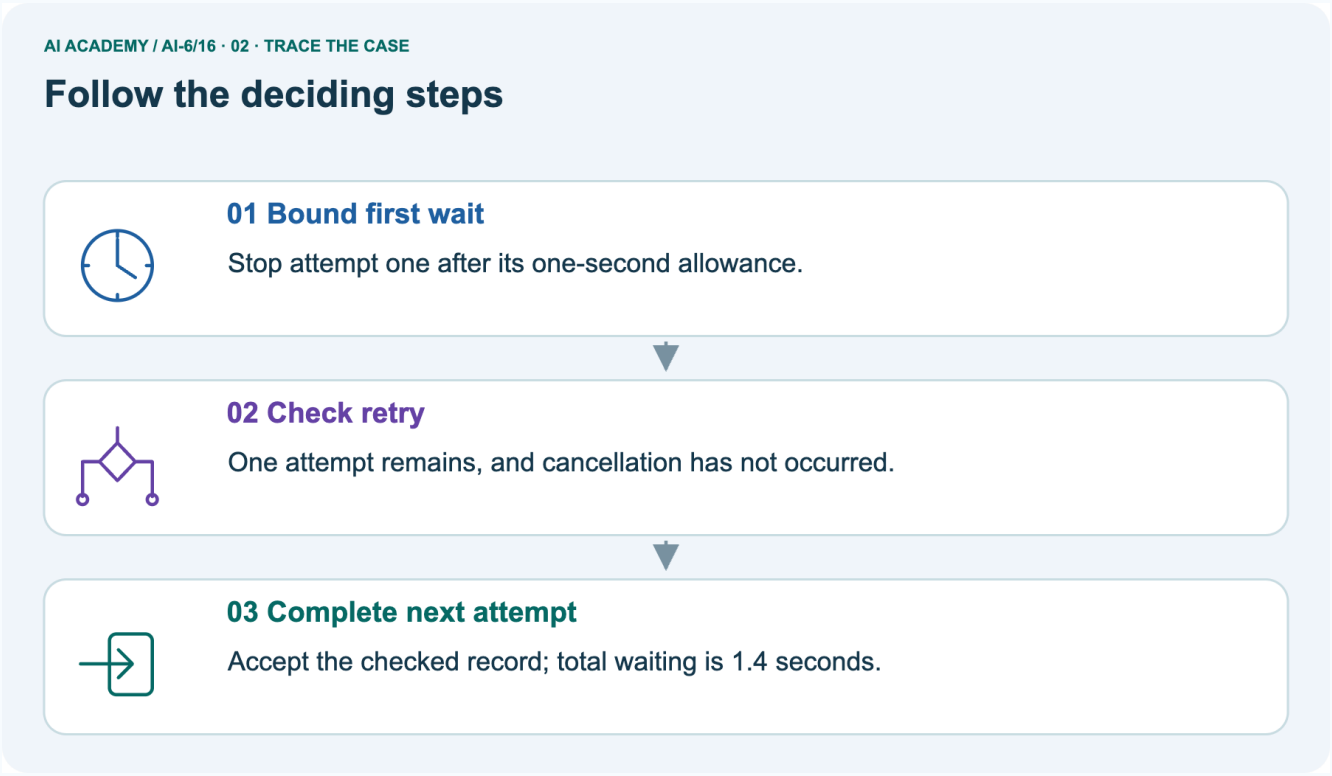
Retry within declared limits. A read-only lookup allows two attempts of at most one second each. **Cancellation** forbids further attempts.

First attempt times out at 1 second. Second succeeds in 0.4 seconds. Ignore overhead. No **cancellation** occurs.

The main model allows 3 attempts with 4 second caller caps and gaps 1/2 seconds. The picture is separate: a read-only lookup allows 2 attempts with 1 second caps, no retry gap and no overhead. A 1-second timeout followed by a 0.4-second successful fixture totals 1.4 seconds.

If cancellation is supplied immediately after the first timeout and before retry/backoff, the second attempt does not start and modeled elapsed waiting stays 1 second. Success is a supplied checked-outcome fixture, not an actual response validation or live lookup.

## Trace the 1-second plus 0.4-second path




Follow the labelled picture with the matching explanation. Open the vector diagram

### Read the picture one step at a time

| STEP | WHAT HAPPENS AND WHY |
| --- | --- |
| 1. Bound first wait | Stop attempt one after its one-second allowance. |
| 2. Check retry | One attempt remains, and cancellation has not occurred. |
| 3. Complete next attempt | Accept the checked record; total waiting is 1.4 seconds. |

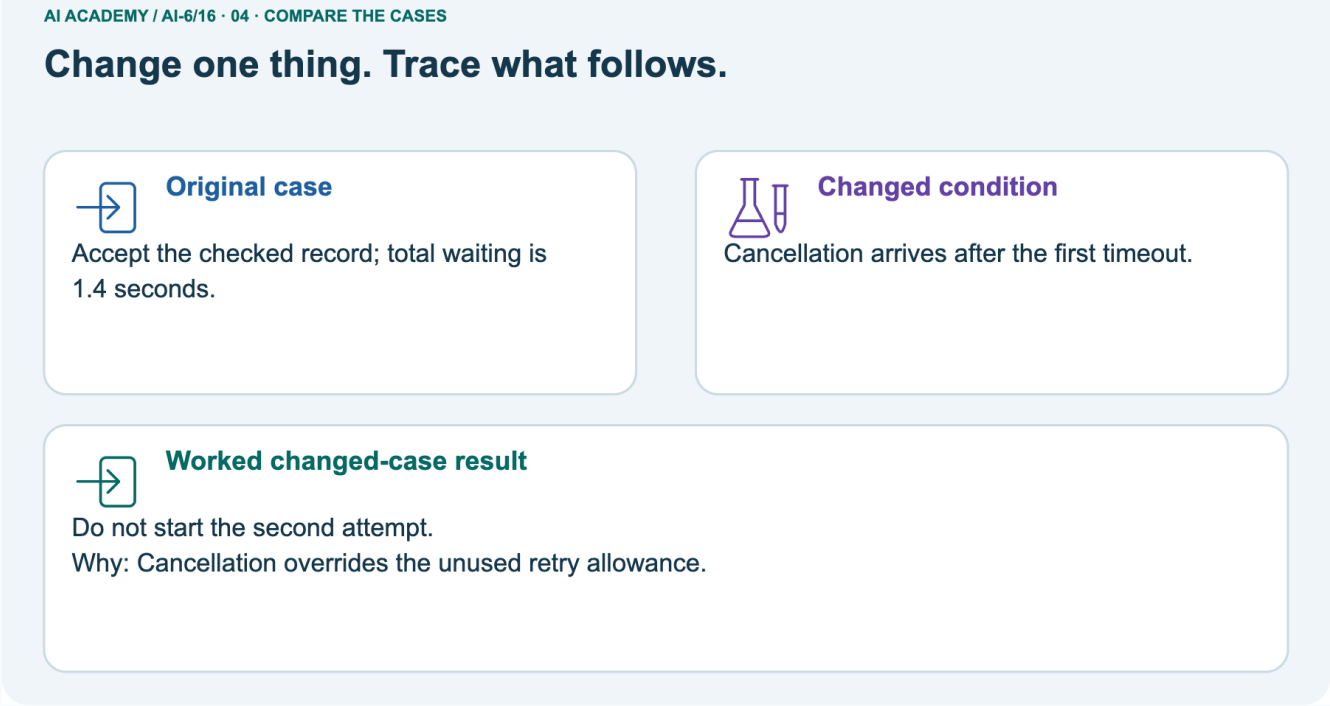
Match each pictured step to the supplied facts. This is a worked teaching case, not an independent assessment.

A **timeout** bounds waiting; a retry tries again; **cancellation** withdraws the request. They are separate conditions on the next action.

## Predict cancellation before attempt 2

- Change one thing: **Cancellation** arrives after the first **timeout**. Predict the result before checking the explanation. What remains the same?

▼ Check after predicting




Follow the labelled picture with the matching explanation. Open the vector diagram

## Explain remote uncertainty and excluded overhead

- The arithmetic excludes overhead; retries of side-effecting operations require separate duplicate and recovery controls.

- Read the labelled picture: Calculate waiting time for a one-second **timeout** followed by a 0.4-second success. What changes if **cancellation** arrives after the first attempt but before the retry?

▼ Reference: original compact bound calculator

Optional local lab: predict the printed result before running this standalone Python 3 example. Change one relevant boundary **input**, record the outcome and state the contract or statistical assumptions.

The program uses fictional values and the Python standard library; it performs no external operations.

```
def waiting_bound(attempts, per_attempt, gaps):
    if type(attempts) is not int or attempts < 1 or len(gaps) != attempts - 1:
        raise ValueError('attempt budget')
    if per_attempt < 0 or any((g < 0 for g in gaps)):
        raise ValueError('negative duration')
    return attempts * per_attempt + sum(gaps)

print(waiting_bound(3, 4, [1, 2]))
```

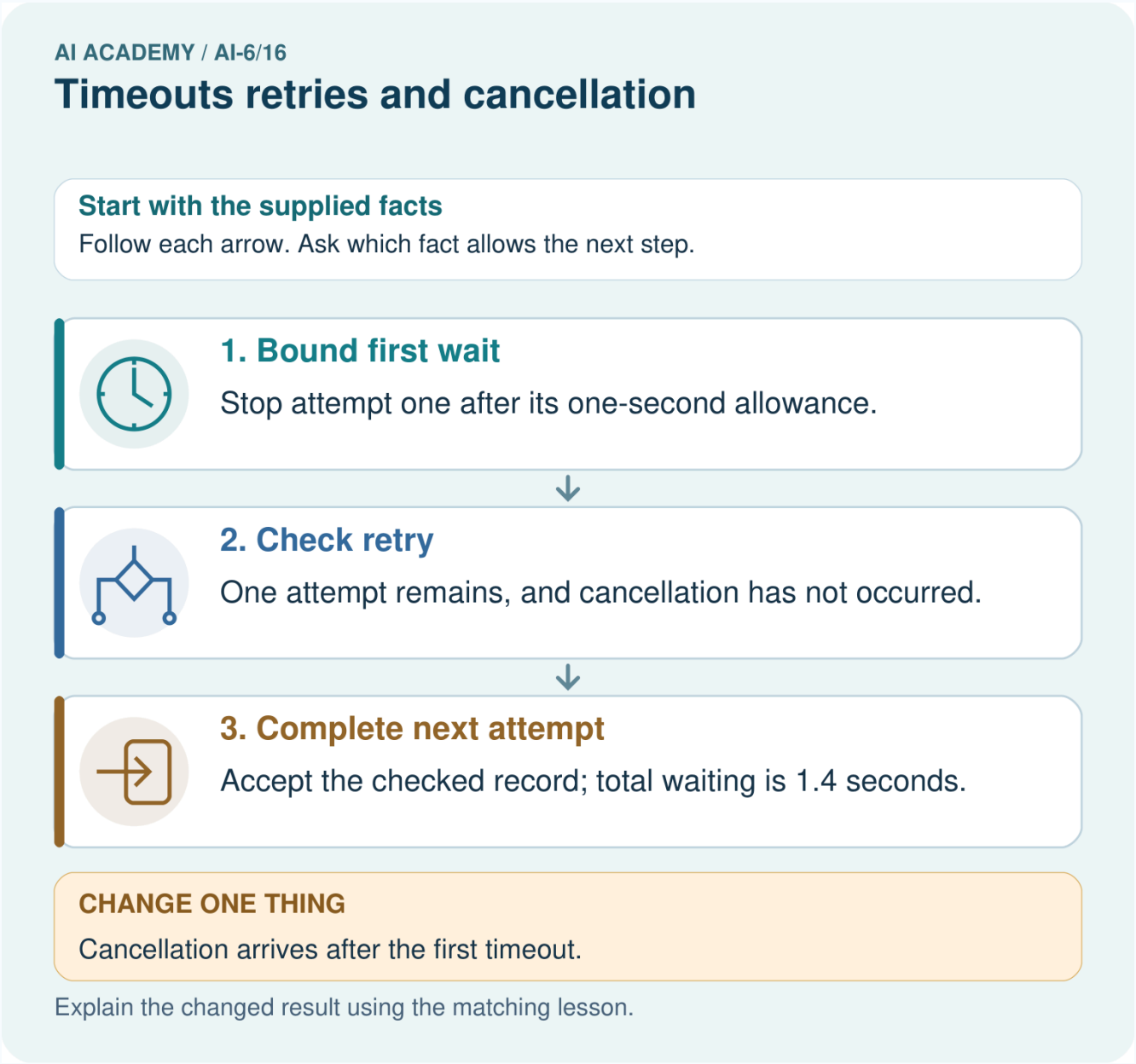
## ▼ Earlier wording, alternatives and picture transcripts

### Learn from the worked case

Cover the result before checking it. Say what is given, choose the procedure, and follow one step at a time. Then uncover the result and explain your first mismatch. Ask why a step is allowed, not only what number it produces.

## A pictorial worked case: Timeouts retries and cancellation

### The facts we will use




Follow the labelled picture with the matching explanation. Open the vector diagram

### Why this result follows

### What this example does not establish

MAIN LESSON · SECTION 7

# Know what stopped: local waiting, future attempts or a remote effect

> Those are different claims with different evidence needs.

## Core timeout-versus-remote-outcome qualification

A **timeout** means the caller lacks a timely result, not necessarily that the remote operation did not occur.

A useful check asks what would make the reasoning fail. Distinguish a failure to execute the declared procedure from a weakness in the procedure itself.

The first may require a correction; the second requires an explicitly revised method and new **evidence**. Preserve unsuccessful cases so an improved result cannot hide the original problem.

## Complete transient, cancellation and numeric-boundary probes

| Timing or retry probe | Expected interpretation |
|---|---|
| Three 4-second timeouts plus gaps 1/2 | 15 seconds modeled waiting; attempt 4 not permitted |
| A declared transient failure with budget remaining | May retry under this local policy after the declared gap |
| Authentication/permission/invalid-request failure | Stop automatic retry; spare budget does not repair authority or input |
| Cancellation before next attempt/backoff | Stop future modeled work without deleting earlier outcome records |
| Negative, nonfinite or Boolean duration | Reject in the extended arithmetic helper |
| Gap count differs from attempts−1 | Reject incomplete timing contract |

| Timing or retry probe | Expected interpretation |
| --- | --- |
| Caller timeout occurs | Remote work may still have happened; reconcile possible committed effects |
| Cancellation is requested | Does not automatically undo already committed effects |
| Virtual simulation passes | No measured wall-clock bound, network abort or asynchronous cancellation implementation established |

## Guided explanation and repair

- A report omits this qualification: A **timeout** means the caller lacks a timely result, not necessarily that the remote operation did not occur.

  Explain how omitting it could change a reader's interpretation, and write an accurate replacement.

**Guided: restate the worked result, then name the assumption or limitation that prevents a broader conclusion.**

**State one repair and the checks you would rerun.**

▼ Earlier wording, alternatives and picture transcripts

**Locate the earliest unsupported step**

---

MAIN LESSON · SECTION 8

# Practise attempts, gaps and stop conditions

> Count the initial attempt and each retry separately.

## Current explanation and guided scope check

- Explain the weekly **mechanism** in your own words. Use the worked scenario to name its **input**, decision rule and **output**.

**Guided: restate the worked result, then name the assumption or limitation that prevents a broader conclusion.**

**Explain your prediction to a partner before comparing results.**

## Supported W2 record

Use workbook W2 for the connected example "**Timeout**s retries and **cancellation**". First fill the **input** and rule boxes. Try the middle steps before asking for a hint. After a hint, try the next step yourself.

| Plan | Do | Check |
| --- | --- | --- |
| What is given? Which rule or procedure applies? | Record each intermediate step. | Does the result follow? What remains unknown? |

Explain your trace to a partner. Your partner points to one step to check. Swap roles on the next case. Each person writes their own explanation.

## Individual remote-outcome limit check

- Use the worked case for **Timeout**s retries and **cancellation**. Proposed explanation: “The worked result establishes every future case”. Decide whether this explanation is supported.

  Point to the step or supplied fact that decides; explain your repair if needed.

- Answer on your own before discussion. If you are unsure, say which step you need help with.

## Cancellation picture response

- Calculate waiting time for a one-second **timeout** followed by a 0.4-second success. What changes if **cancellation** arrives after the first attempt but before the retry?

> **My explanation using a pictured step and the given facts**
>
> _____
>
> _____
>
> _____
>
> _____

## Reserve independent two-attempt/authentication task

This is supported practice. Keep the separately named independent case for an attempt before feedback.

▼ Earlier wording, alternatives and picture transcripts

**Practise with less help: Trace the worked case**

**Check your reasoning**

**Explain the picture**

---

MAIN LESSON · SECTION 9

# Run a complete virtual-time and stop-condition audit

> Inspect the event list and timing assumptions rather than claiming a real timeout handler.

## Current three-timeout/fourth-proposal investigation

Three attempts each time out after 4 seconds with gaps 1 and 2. A fourth is proposed. Show the intermediate steps and give the conclusion with its scope.

## Complete runnable virtual model and boundary checks

```
"""Virtual caller-wait arithmetic only: no sleep, network, remote abort or undo."""
from math import isfinite,isclose

def duration(value):
    return type(value) in (int,float) and isfinite(value) and value>=0

def waiting_bound(attempts,per_attempt,gaps):
    if type(attempts) is not int or attempts<1 or len(gaps)!=attempts-1:
        raise ValueError('invalid attempt budget/gap count')
    if not duration(per_attempt) or any(not duration(g) for g in gaps):
        raise ValueError('durations must be finite and nonnegative')
    return attempts*per_attempt+sum(gaps)

def simulate(outcomes,attempts,per_attempt,gaps,cancel_before_attempt=None):
    bound=waiting_bound(attempts,per_attempt,gaps)
    if cancel_before_attempt is not None and (type(cancel_before_attempt) is not int or
not 1<=cancel_before_attempt<=attempts):
        raise ValueError('invalid cancellation boundary')
```

```
    elapsed=0;events=[]
    for index in range(attempts):
        number=index+1
        # This model only places cancellation before a next attempt/backoff begins.
        if number==cancel_before_attempt:
            return
{'status':'cancelled','elapsed_s':elapsed,'events':events,'bound_s':bound}
        if index:
            elapsed+=gaps[index-1]
            events.append({'kind':'gap','seconds':gaps[index-1],'elapsed_s':elapsed})
        if index>=len(outcomes):raise ValueError('missing supplied outcome fixture')
        waited,status=outcomes[index]
        if (not duration(waited) or waited>per_attempt or
            status not in
('timeout','transient','success','auth_error','permission_error','invalid_request')):
            raise ValueError('invalid caller-wait outcome')
        if status=='timeout' and waited!=per_attempt:
            raise ValueError('timeout fixture must reach its declared wait cap')
        elapsed+=waited
        events.append({'kind':'attempt','number':number,'status':status,
                       'caller_wait_s':waited,'elapsed_s':elapsed})
        if status=='success':return
{'status':'success','elapsed_s':elapsed,'events':events,'bound_s':bound}
        if status not in ('timeout','transient'):
            return
{'status':'nonretryable','elapsed_s':elapsed,'events':events,'bound_s':bound}
    return
{'status':'budget_exhausted','elapsed_s':elapsed,'events':events,'bound_s':bound}

main=simulate([(4,'timeout')]*3+[(.1,'success')],3,4,[1,2])
assert main['elapsed_s']==15 and main['status']=='budget_exhausted'
assert len([e for e in main['events'] if e['kind']=='attempt'])==3
print('Main virtual wait trace:',main)
picture=simulate([(1,'timeout'),(.4,'success')],2,1,[0])
cancelled=simulate([(1,'timeout'),(.4,'success')],2,1,[0],cancel_before_attempt=2)
assert picture['status']=='success' and isclose(picture['elapsed_s'],1.4)
assert cancelled['status']=='cancelled' and cancelled['elapsed_s']==1
assert len([e for e in cancelled['events'] if e['kind']=='attempt'])==1
print('Separate picture without cancellation:',picture)
print('Cancellation after first timeout:',cancelled)
nonretryable=simulate([(.2,'permission_error'),(.2,'success')],3,4,[1,2])
assert nonretryable['status']=='nonretryable' and nonretryable['elapsed_s']==.2
assert len(nonretryable['events'])==1
print('Non-transient failure stops immediately:',nonretryable)
print('Timeout/cancellation of caller waiting does not prove remote work stopped or a
committed effect was undone.')
```

```
print('Only declared boundary cancellation is modeled; no real asynchronous cancellation,
overhead or wall-clock measurement.')
```

## Expected elapsed totals, consumed attempts and limits

1. Predict the main event timeline before running or tracing. The program performs no sleep or network call: it adds supplied caller-wait durations and gaps. It consumes at most 3 attempt outcomes even when a fourth fixture is available. A separate two-attempt picture succeeds after 1.4 virtual seconds; cancellation before the second attempt stops at 1 second.

   A non-transient permission error stops after its first supplied outcome. The extended helper rejects nonfinite, negative and Boolean durations and incorrect gap counts. Cancellation is modeled only at declared between-attempt boundaries, not during a real asynchronous operation. Preserve possible late-effect uncertainty and the separate approval/idempotency/recovery controls for side effects.

## Evidence ledger and reflection

| Record | Your evidence |
| --- | --- |
| Given input and fixed rule | |
| Prediction before checking | |
| Actual result, or “not run” | |
| First mismatch and one explanation | |
| Boundary or missing input and expected response | |

On paper, trace the supplied procedure. If code is provided, predict before running it in a local practice file. Never write an expected answer as though it were an observed run.

Before trying: what do I expect, and why? While trying: which step could first go wrong? Afterwards: what did I actually observe, and what should change? Keep “not run” if you only traced the procedure.

### ▼ Reference: original compact duration helper

```
def waiting_bound(attempts, per_attempt, gaps):
    if type(attempts) is not int or attempts < 1 or len(gaps) != attempts - 1:
        raise ValueError('attempt budget')
    if per_attempt < 0 or any((g < 0 for g in gaps)):
        raise ValueError('negative duration')
    return attempts * per_attempt + sum(gaps)

print(waiting_bound(3, 4, [1, 2]))
```

### ▼ Earlier wording, alternatives and picture transcripts

**Predict, investigate and preserve the evidence**

**Plan → try → check**

---

MAIN LESSON · SECTION 14

# Try alone: calculate a new bound and classify the failure

> Separate arithmetic budget from permission to retry authentication failures.

### Complete independent two-attempt scenario

Two attempts, each up to 5 seconds, gap 2. Compute bound and decide whether authentication failure should retry.

## First attempt and support record

### My independent explanation, trace and limits

Attempt this case before feedback. Record any help. Earlier examples below are further practice; a case already practised with feedback cannot establish fresh transfer.

## Feedback after trying

Attempt the independent case before hints. Record whether you worked alone, used a hint or followed a model. After feedback, fix the first unsupported step and explain why it changed.

A later check uses a changed case, not a copied answer.

### ▼ Earlier wording, alternatives and picture transcripts

#### Independent case: Apply a new case independently

#### Attempt this without hints

Two attempts, each up to 5 seconds, gap 2. Compute bound and decide whether authentication failure should retry.

### My independent procedure, result and limitation

Record support used: none, hint or teacher explanation. After feedback, a different case is needed to demonstrate independence.

#### Use feedback to improve

MAIN LESSON · SECTION 11

# Save timeout scope, retry categories and cancellation semantics together

> Keep attempt counts, gaps, event traces and uncertain remote outcomes explicit.

## Project task and artifact

Add a labelled **timeout**s retries and **cancellation evidence** sheet to Grounded Learning Service. Show how you can explain the **mechanism** and terminology of **timeout**s retries and **cancellation**.

Use the supplied fictional case and keep its **input**s, intermediate steps, calculation or **prediction**, and limitation. Connect this record to last week's record; a matching answer without a trace is insufficient.

| Project evidence | Record |
| --- | --- |
| Component and version | |
| Inputs permitted at prediction time | |
| Trace and comparison with earlier work | |
| Failure, limit and human review | |

## Actual run or trace and limitations

For the Grounded Learning Service, retain an annotated trace and the changed-case result. Name the contract or **evidence** that makes the deciding step valid, so another team can review it.

▼ Earlier wording, alternatives and picture transcripts

**Evidence for your project**

---

MAIN LESSON · SECTION 16

# Finish with a bounded wait claim and an explicit stop rule

> A local stop does not prove remote rollback or future service performance.

## Retrieve and explain

- Close your notes. Explain this goal in your own words: Explain the **mechanism** and terminology of **timeout**s retries and **cancellation**

> **My deciding step, evidence and one remaining question**
>
> | |
> |---|
> | |
> | |
> | |
> | |

## Plan a changed-case return

If you are unsure, return to the worked picture and explain one arrow. If you are ready, change an **input** or assumption and justify your **prediction**.

Save your workbook **evidence**. At the next review, try a different case before opening your old answer.

# Reference bank and extra practice

Choose the reference that helps your next step.

▼ **Reference · Use service vocabulary precisely**

REFERENCE · SECTION 5

## Reference · Use service vocabulary precisely

> Connect each term to an input, operation, evidence source or outcome.

### Vocabulary and your explanation

| Word | Meaning |
| --- | --- |
| Timeout | A declared limit on waiting |
| Retry budget | The maximum repeat attempts or resources |
| Cancellation | A request to stop future processing |

**Use two terms to explain a step in this week's example.**

▼ Extra challenge · Explain a stronger case

REFERENCE · SECTION 10

# Extra challenge · Explain a stronger case

> Choose the next challenge after the main route.

## Explain,repair and extend

- Explain the weekly **mechanism** in your own words. Use the worked scenario to name its **input**, decision rule and **output**.

- A report omits this qualification: A **timeout** means the caller lacks a timely result, not necessarily that the remote operation did not occur.

  Explain how omitting it could change a reader's interpretation, and write an accurate replacement.

- For the worked scenario, propose one additional case that could reveal a failure, describe its expected outcome before running it, and explain what would need to remain fixed for a fair comparison.

  Do not reuse W4 as a new final test after receiving feedback.

### Explain → apply → challenge

> **Explain which evidence supports your conclusion and which claim remains unproved.**

▼ Reference · Check the service reasoning and its limits

REFERENCE · SECTION 12

# Reference · Check the service reasoning and its limits

> Use after practice; keep the selected independent case for its first attempt.

## Current explanation and response

- Complete the supported workbook tasks before attempting W4 independently. Record any hints or help used. After feedback, use the teacher's fresh retry rather than copying the earlier answer.

  For a project, select a relevant new case, predict the outcome from the declared rule and preserve the result alongside any revision.

- For the worked scenario, propose one additional case that could reveal a failure, describe its expected outcome before running it, and explain what would need to remain fixed for a fair comparison.

  Do not reuse W4 as a new final test after receiving feedback.

**Exit: propose one revealing change to the worked scenario. State the expected outcome and the evidence that would make you revise your explanation.**

**My limited claim and a useful next check**

▼ **Extra practice · Revisit the complete trace**

REFERENCE · SECTION 13

## Extra practice · Revisit the complete trace

> Predict before reading the supplied steps.

## Complete trace and evidence

### Complete evidence record

#### Timeouts retries and cancellation — evidence trace

| INPUT AND SCOPE | CHECK OR CALCULATION | CONCLUSION AND LIMIT |
| --- | --- | --- |
| Three attempts each time out after 4 seconds with gaps 1 and 2. A fourth is proposed. | Retry only declared transient failures. Use at most three attempts with waiting gaps 1 and 2 seconds; the caller waits at most 4 seconds for each attempt under the enforced local timeout model. Worst-case elapsed bound excluding overhead is 3*4+1+2=15 seconds. | Budget is exhausted after 15 seconds under the model; reject fourth attempt. A late side-effect result must still be reconciled. |

The figure connects this chapter's worked input, declared procedure and result. Read the complete worked reasoning for qualifiers and intermediate steps; the abbreviated labels do not replace that reasoning.

Follow the **input** through the stated procedure to its result. Cover the result first, predict it, then explain every intermediate step. The text/table accompanies the figure so the example can be read without colour.

**My complete trace of the worked case**

▼ **Reference · The earlier route checklist**

REFERENCE · SECTION 15

# Reference · The earlier route checklist

> Use one main route rather than repeating the lesson.

## Targets,route and first prediction

Start with this question: Retry within declared limits. A read-only lookup allows two attempts of at most one second each. **Cancellation** forbids further attempts.

| By the end, I can | How I will show it |
|---|---|
| Explain the mechanism and terminology of timeouts retries and cancellation | Explain the deciding step, show a trace or test, and state one limit. |
| Apply the stated procedure to a new case with a traceable result | Explain the deciding step, show a trace or test, and state one limit. |
| Identify an assumption or limitation and design a revealing follow-up | Explain the deciding step, show a trace or test, and state one limit. |

Remember → watch and explain → try with help → investigate → try independently → keep project **evidence**. Your teacher may spread this lesson across several classes.

Keep your first **prediction**. Compare it with what the **evidence** shows and explain any change.

▼ Extra practice · Predict,check and repair

REFERENCE · SECTION 17

# Extra practice · Predict,check and repair

> Keep supported feedback separate from the independent case.

## First supported question and optional hint

- Retry within declared limits. A read-only lookup allows two attempts of at most one second each. **Cancellation** forbids further attempts.
- Which labelled step matches this detail? Accept the checked record; total waiting is 1.4 seconds.
- 1. Complete next attempt
- 2. Bound first wait
- 3. Check retry
- Hint if needed: Read the steps in the pictorial case. Match the supplied detail to the stage that performs or records it.

### Practice 1: explain the deciding evidence

**My selected conclusion, deciding fact and explanation**

▼ **Check after the first attempt**

After your attempt, compare with the supported explanation: Complete next attempt Accept the checked record; total waiting is 1.4 seconds. A **timeout** bounds waiting; a retry tries again; **cancellation** withdraws the request.

They are separate conditions on the next action.

## Changed-case question and optional hint

- Changed-case practice: **Cancellation** arrives after the first **timeout**. Which conclusion is supported by the supplied facts?
- 1. We can decide the result without examining the changed **input** or the procedure.
- 2. Do not start the second attempt.
- 3. This one example guarantees correct results for every future **input**.
- Hint if needed: Use the changed condition and the deciding step in the pictorial trace. The answer must say what follows from those facts.

### Practice 2: explain the deciding evidence

**My selected conclusion, deciding fact and explanation**

▼ **Check after the changed attempt**

After your attempt, compare with the supported explanation: Do not start the second attempt. **Cancellation** overrides the unused retry allowance.

## Repair and retain the record

Review Approval gates and idempotency (ai-6/15). Goal: Explain the **mechanism** and terminology of approval gates and idempotency. Short practice: Trace the two deliveries of approved K9.

How many reservations should occur, and why can a new K10 for desk 5 not borrow K9’s approval?. Return to this week and explain what you changed.

Review State machines for bounded agents (ai-6/13). Goal: Explain the **mechanism** and terminology of state machines for bounded agents. Short practice: Trace A and B through the state diagram.

Name B’s forbidden transition and explain whether adding checking→querying permits that shortcut.. Return to this week and explain what you changed.

Review Evaluation pipelines (ai-6/10). Goal: Explain the **mechanism** and terminology of evaluation pipelines. Short practice: Calculate A’s and B’s correct rates on all four cases.

Identify their shared failure and explain why removing that case after scoring changes the honesty of the comparison.. Return to this week and explain what you changed.

The browser lab lets you edit the supported Python examples already included in this lesson. Predict first, run, inspect the actual **output** or error, change one **input**, then run again.

Keep code, **prediction**s, actual results and a reasoned revision together. On paper, label your result as a trace rather than an execution.

Use Download completed work in the hosted lesson to export your answers, reflections, practice feedback and code-run **evidence**. Open the HTML copy to read or print it.

Drafts stay in this browser when storage is available; clear drafts on a shared device.

▼ **Earlier wording, alternatives and picture transcripts**

Use this supported practice once, in the browser or on paper. Keep the reserved independent assessment for a separate first attempt before teacher feedback.

**Repair a missing building block**

**Predict, run and debug**

**Keep your completed work**

▼ Reference · Six-picture concept journey

REFERENCE · SECTION 18

# Reference · Six-picture concept journey

> Choose the picture that supports the current explanation.

## 01. Timeouts retries and cancellation: the case



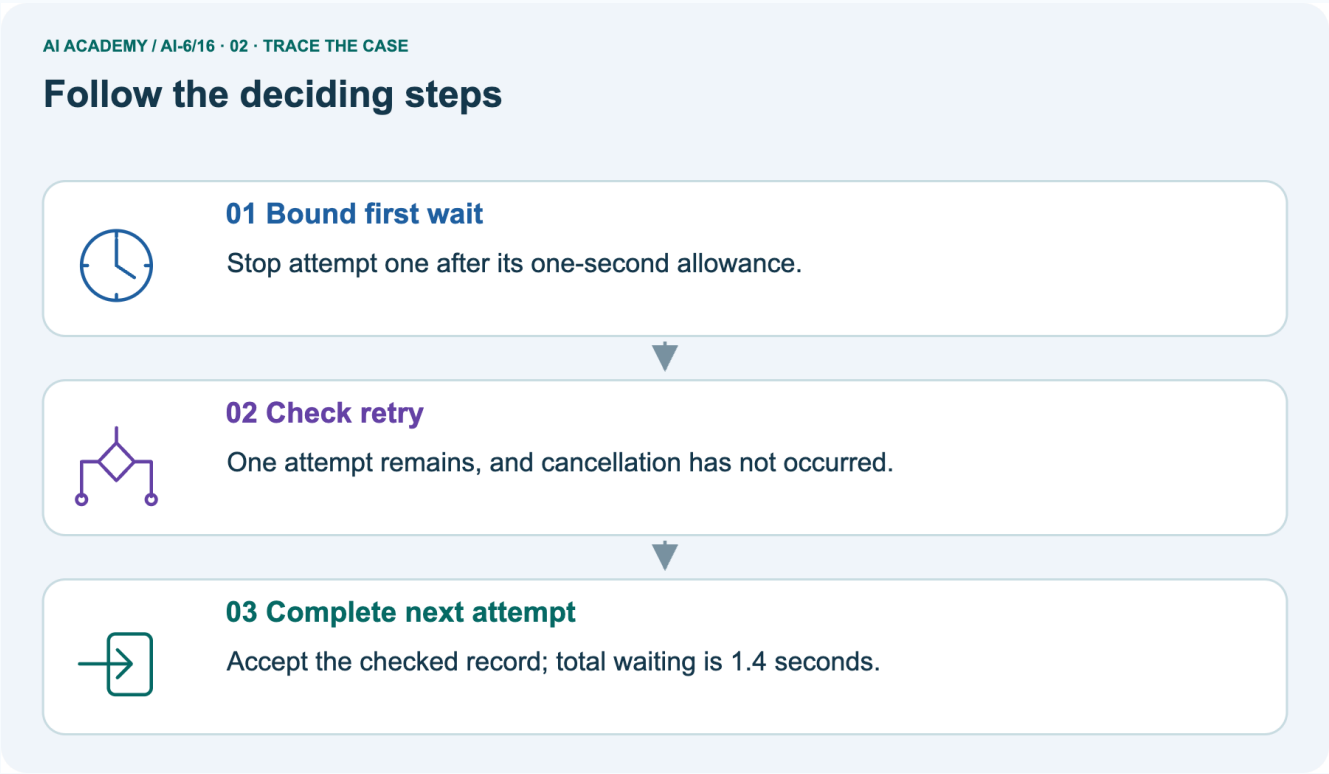

Follow the labelled picture with the matching explanation. Open the vector diagram

Retry within declared limits. A read-only lookup allows two attempts of at most one second each. **Cancellation** forbids further attempts. Supplied facts: First attempt times out at 1 second. Second succeeds in 0.4 seconds.

Ignore overhead. No **cancellation** occurs.

Point to the given fact that matters most. Explain why it matters.

## 02. Follow the deciding steps

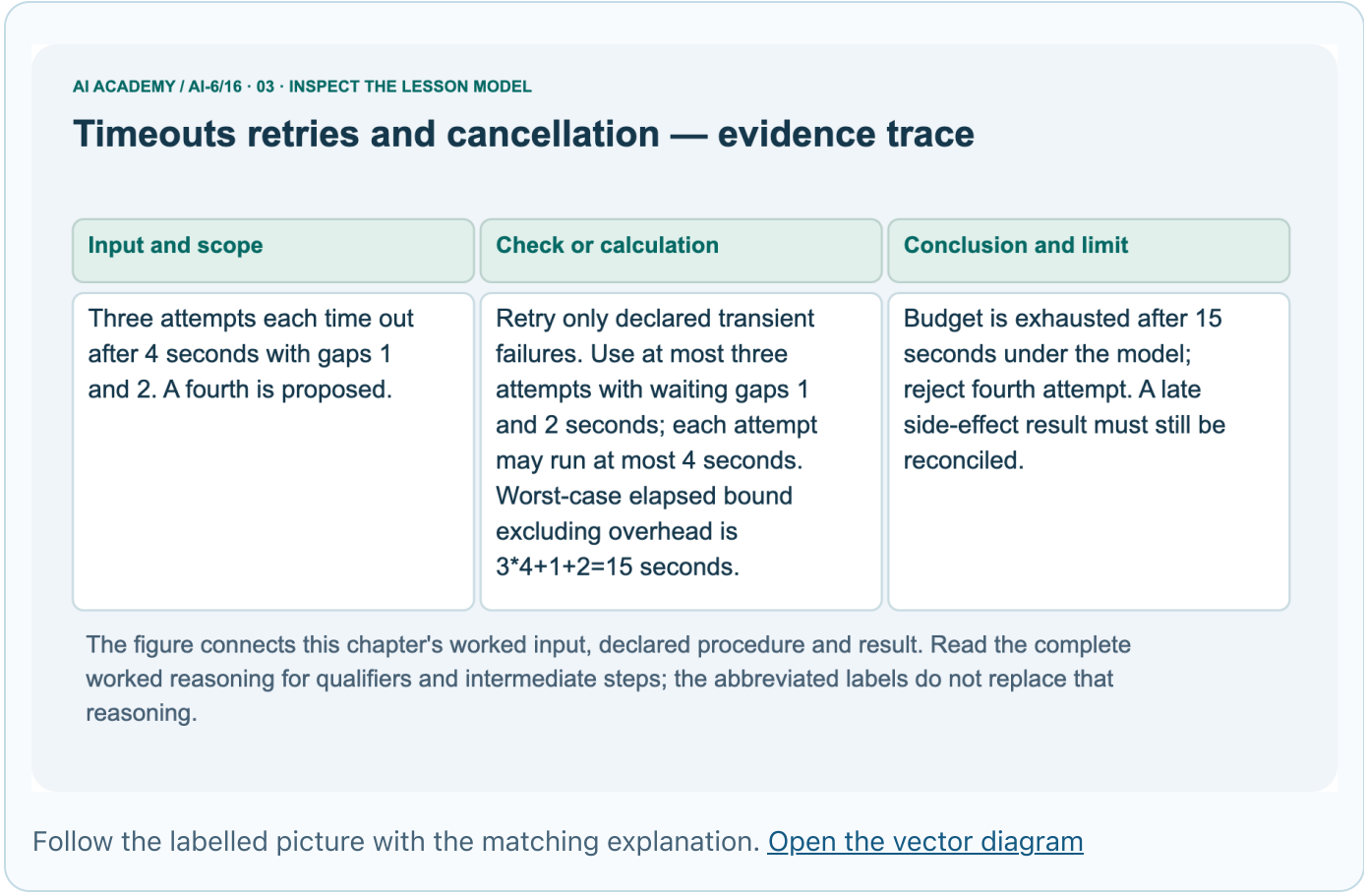



Follow the labelled picture with the matching explanation. Open the vector diagram

Bound first wait: Stop attempt one after its one-second allowance. → Check retry: One attempt remains, and **cancellation** has not occurred. → Complete next attempt: Accept the checked record; total waiting is 1.4 seconds.

Point to each numbered stage and explain which recorded **input** it uses.

## 03. Timeouts retries and cancellation — evidence trace

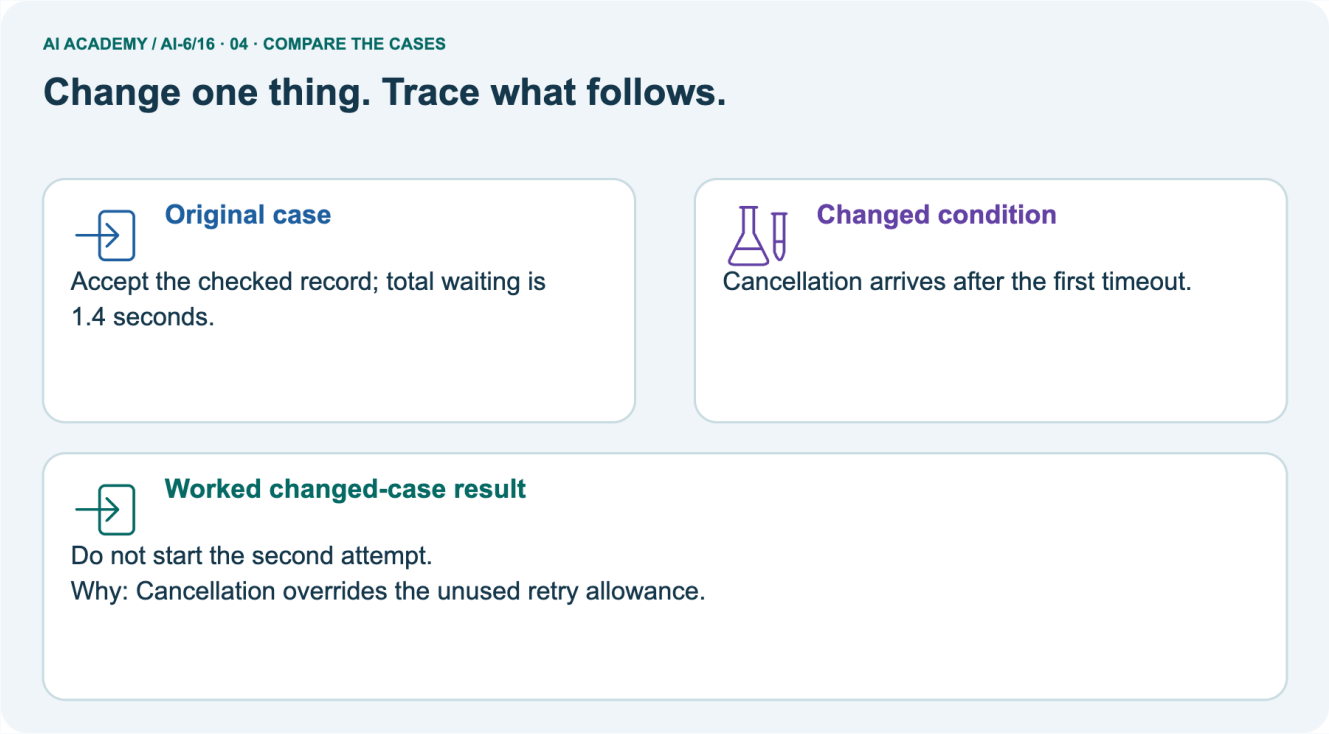



Follow the labelled picture with the matching explanation. Open the vector diagram

**Timeout**s retries and **cancellation** — **evidence** trace. Three attempts each time out after 4 seconds with gaps 1 and 2. A fourth is proposed. | Retry only declared transient failures. Use at most three attempts with waiting gaps 1 and 2 seconds; the caller waits at most 4 seconds for each attempt under the enforced local **timeout** model. Worst-case elapsed bound excluding overhead is 3*4+1+2=15 seconds. | Budget is exhausted after 15 seconds under the model; reject fourth attempt. A late side-effect result must still be reconciled.

Read the labels and values. Explain the relationship the model shows.

## 04. Change one thing. Trace what follows.



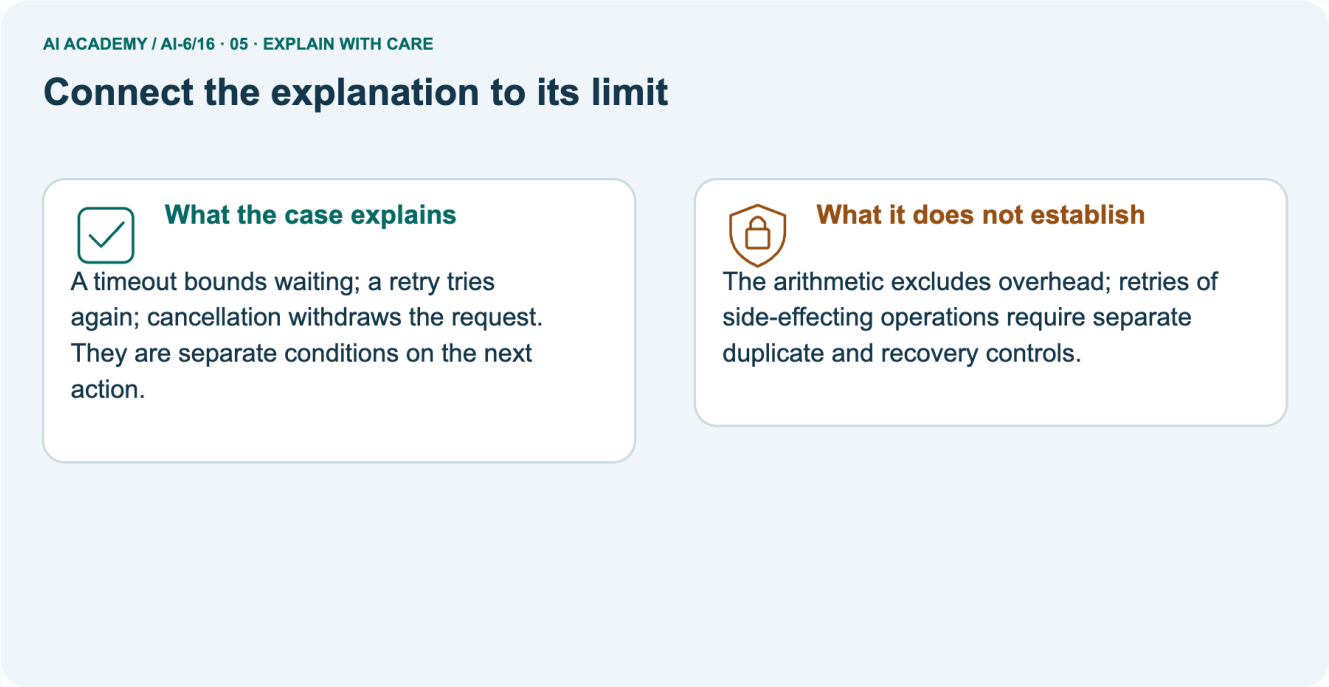

Follow the labelled picture with the matching explanation. Open the vector diagram

Original case: Accept the checked record; total waiting is 1.4 seconds. Changed condition: **Cancellation** arrives after the first **timeout**. Expected result: Do not start the second attempt.

Reason: **Cancellation** overrides the unused retry allowance.

Predict before checking the worked changed-case result. Name what changed and what the supplied facts allow you to conclude.

## 05. Connect the explanation to its limit



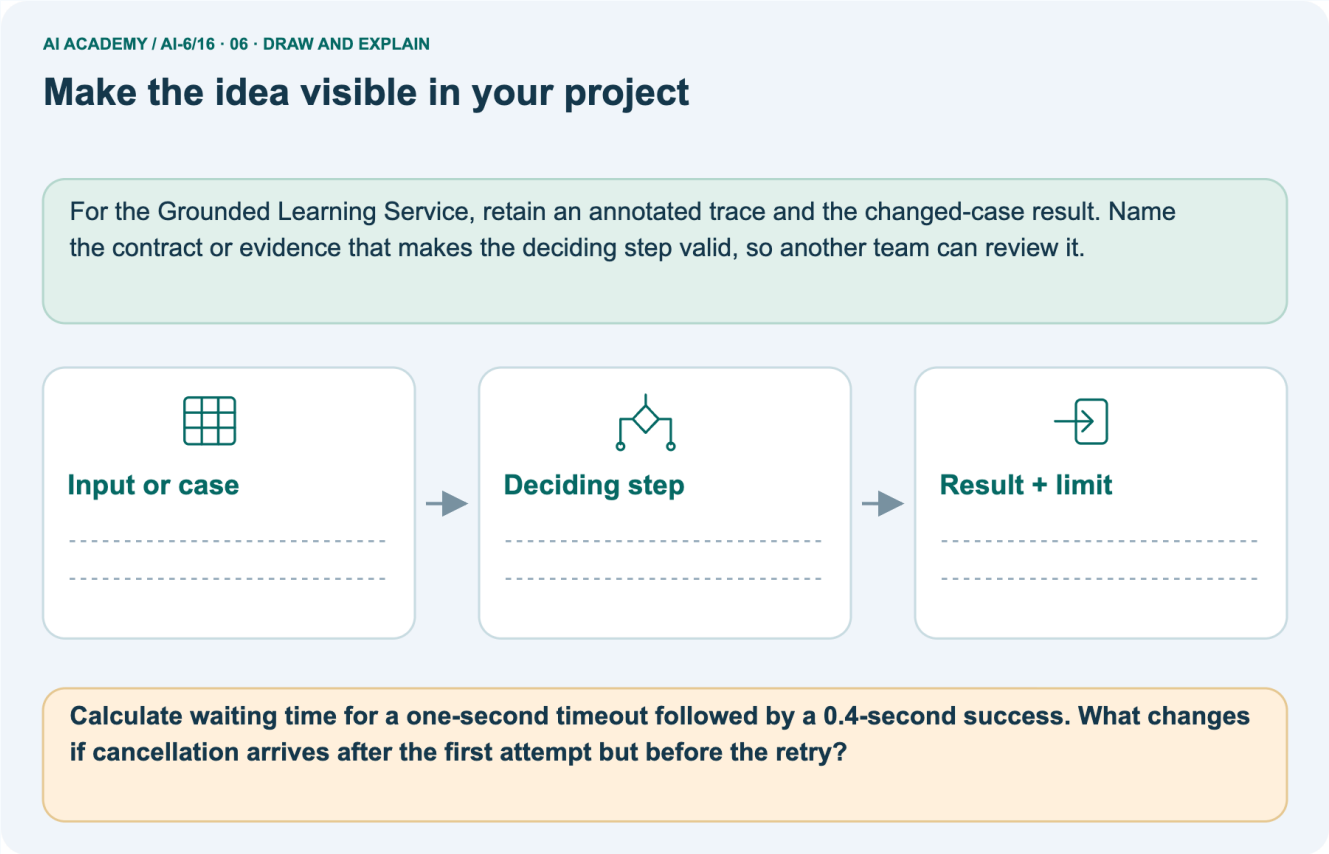

Follow the labelled picture with the matching explanation. Open the vector diagram

A **timeout** bounds waiting; a retry tries again; **cancellation** withdraws the request. They are separate conditions on the next action.

Limit: The arithmetic excludes overhead; retries of side-effecting operations require separate duplicate and recovery controls.

Explain the case, then point to a limit on the claim.

## 06. Make the idea visible in your project




Follow the labelled picture with the matching explanation. Open the vector diagram

For the Grounded Learning Service, retain an annotated trace and the changed-case result. Name the contract or **evidence** that makes the deciding step valid, so another team can review it.

Draw your **input**, deciding step, result and limit. Supported question: Calculate waiting time for a one-second **timeout** followed by a 0.4-second success.

What changes if **cancellation** arrives after the first attempt but before the retry?

Draw your project trace and justify the deciding step using **evidence**.

## ▼ Earlier wording, alternatives and picture transcripts

**01. Timeouts retries and cancellation: the case**

**02. Follow the deciding steps**

**03. Timeouts retries and cancellation — evidence trace**

**04. Change one thing. Trace what follows.**

**05. Connect the explanation to its limit**

**06. Make the idea visible in your project**

## ▼ Optional interactive practice and completed work

Save or print your responses for your portfolio. Progress and drafts on the hosted website stay in the current browser. Use only fictional or teacher-approved data.